

Laboratorio 2:

Unidad Curricular: Tecnologías de Microprocesamiento
Universidad Tecnológica (UTEC) - Ingeniería Mecatrónica

Tizziano Travers (C.I.: 5.623.448-1)
Bruno Bentancourt (C.I.: 5.495.926-3)
Giuliano Gilardoni (C.I.: 5.644.255-9)
Docentes: Jesús Martínez / Marcelo Díaz

I. INTRODUCCIÓN

El presente informe documenta el diseño, la simulación y la implementación física de cinco sistemas embebidos basados en el microcontrolador ATmega328P programados en lenguaje ensamblador AVR. En primer lugar, se desarrolló el control de una matriz de LEDs de 8×8 mediante multiplexación por barrido de filas, conmutación de modos por consola serie (UART) y ajuste de velocidad de desplazamiento. En segundo lugar, se implementó un generador de funciones utilizando un convertidor digital-analógico (DAC) con red R-2R de 8 bits para sintetizar formas de onda almacenadas en tablas de búsqueda (LUT) en la memoria Flash. En tercer lugar, se diseñó el sistema de control digital para un plotter mediante comunicación UART y rutinas de movimiento XY. En cuarto lugar, se estableció un enlace de comunicación USART unidireccional entre dos microcontroladores para transmitir combinaciones de 3 bits y controlar la activación de LEDs. Finalmente, se automatizó el control de una puerta de garaje mediante una máquina de estados finitos (FSM), reporte de eventos vía USART e interrupciones por cambio de pin (PCINT0) para reversa de seguridad. Todos los desarrollos fueron validados experimentalmente en entornos de simulación (PICSimLab y Proteus) y en montajes físicos sobre protoboard.

II. OBJETIVO

Objetivo General Diseñar e implementar firmware en lenguaje ensamblador AVR para el microcontrolador ATmega328P, integrando el control de periféricos digitales y analógicos, protocolos de comunicación serie y sistemas de interrupción en aplicaciones embebidas. **Objetivos Específicos** Implementar el control multiplexado de una matriz de LEDs de 8×8 con texto desplazable dinámico y patrones estáticos seleccionables mediante UART y pulsadores. Desarrollar un generador de funciones sintéticas mediante una red DAC R-2R de 8 bits y tablas de búsqueda (LUT) almacenadas en memoria Flash, con ajuste dinámico de frecuencia. Diseñar las rutinas de movimiento y posicionamiento digital para un plotter controlado por comandos serie UART y actuadores de lápiz. Establecer un enlace de comunicación punto a punto por interfaz USART entre dos microcontroladores para la transmisión y decodificación de datos de 3 bits. Automatizar un sistema de puerta de garaje gobernado por una FSM con supervisión en tiempo real por USART e interrupciones por cambio de pin (PCINT0) para seguridad ante obstáculos.

III. MATERIALES

Para el desarrollo de las diferentes actividades del laboratorio se utilizaron los siguientes componentes:

- Microcontrolador ATmega328P.
- Placa Arduino Uno (Lafvin).
- Matriz de LEDs 8×8 (Modelo 1088AS).
- Protoboard, LEDs, resistencias y pulsadores.
- Cables de conexión.

Arduino Uno

Placa de desarrollo basada en microcontrolador que permite programar y controlar circuitos electrónicos mediante entradas y salidas digitales y analógicas.



Figura 1. Arduino Uno.

ATmega328P

Microcontrolador de 8 bits de la familia AVR. Componente encargado de ejecutar las instrucciones del programa y controlar los periféricos del sistema.

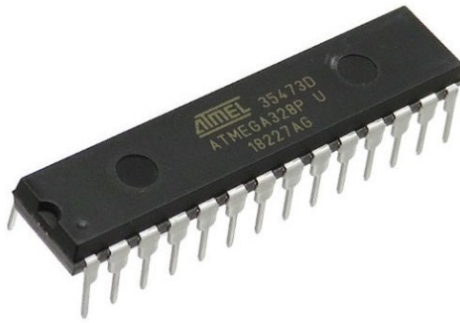


Figura 2. Microcontrolador ATmega328P.

Matriz de LEDs 8x8 1088AS

Panel de 64 LEDs organizados en 8 filas y 8 columnas. Funciona mediante multiplexación a alta velocidad para formar texto y figuras.



Figura 3. Matriz de LEDs 8x8 1088AS.

IV. PROBLEMA A: MATRIZ DE LEDS

IV-A. Objetivo

Diseñar e implementar un programa en lenguaje ensamblador para el microcontrolador ATmega328P que controle una matriz de LEDs de 8×8 , permitiendo el despliegue dinámico de mensajes de texto desplazables y patrones estáticos predefinidos. El sistema busca integrar el barrido por multiplexación de filas/columnas, el manejo de arreglos en memoria Flash/SRAM, y la interacción con el usuario mediante un menú por comunicación serie (UART) y pulsadores físicos.

IV-B. Metodología

La solución desarrollada en ensamblador AVR para el microcontrolador ATmega328P gestiona el despliegue en la matriz de LEDs mediante multiplexado por barrido de filas, conmutación de modos serie y control de velocidad en tiempo real.

IV-C. Máquina de Estados Finitos (FSM)

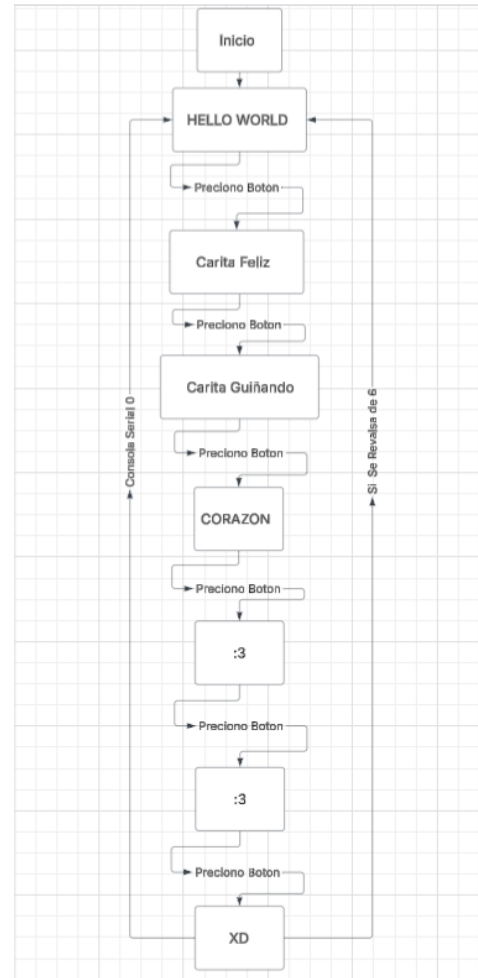


Figura 4. Diagrama de Flujo y Máquina de Estados Finitos (FSM).

IV-C1. Descripción de Estados:

- **Inicio (Reset / Inicialización):** Configura el *Stack Pointer*, inicializa los puertos de E/S, envía el menú mediante el módulo UART y limpia el buffer en RAM.
- **Estado 0 (MODE = 0) – HELLO WORLD (Texto Desplazado):** Modo dinámico donde las columnas de texto se leen secuencialmente desde la memoria Flash (MSG_TEXT_COLS) y se desplazan de derecha a izquierda en el buffer ROW_BUF de la RAM. La rutina CHECK_UART permite ajustar la velocidad mediante las teclas + y -.
- **Estados 1 al 6 (MODE = 1 . . 6) – Patrones Estáticos:** Carga y despliega gráficos fijos leídos desde la tabla PATTERNS en Flash (Carita feliz, guiño, corazón, carita :3, asterisco y la expresión “XD”) mediante multiplexado por fila (~ 1 ms). Al superar el Estado 6, la máquina reinicia a MODE = 0.

Eventos de Transición:

- **Pulsadores Físicos (PC3/PC4):** Incrementan o decrementan el modo activo (MODE) incorporando detección de flanco para evitar rebotes.

- **Consola Serial (UART):** La recepción de caracteres ASCII ('0' a '6') fuerza la transición directa al estado correspondiente.

Implementación Técnica y Justificación de Diseño

Gestión de Memoria y Mapeo de Registros: El diseño asigna funciones específicas a los registros de trabajo (R_{16} a R_{25}). La memoria SRAM alberga el buffer dinámico de 8 bytes (ROW_BUF) para el texto activo, mientras que la memoria Flash almacena las tablas de consulta de patrones e íconos delimitadas por 0xFF.

Cuadro I
ASIGNACIÓN DE REGISTROS DEL SISTEMA

Reg.	Etiqueta	Función / Propósito
R16	TEMP	Registro auxiliar de E/S / Configuración
R17	ROW_IDX	Índice de fila actual (0 a 7)
R18	MODE	Modo activo (0: Scroll, 1 – 6: Estático)
R19	BTN_STATE	Antirebote de pulsadores
R20	DELAY_CNT	Contador de retardo por software
R21	ZERO	Cero constante (0x00) para adc
R22	PAT_BYTE	Datos de columnas de la fila actual
R23	SCROLL_CNT	Contador de cuadros de refresco
R24	SPEED_VAL	Umbral de velocidad (ajustable +/–)
R25	MSG_PTR	Puntero a la columna actual en Flash

Multiplexación por Barrido y Blanking: El refresco por división de tiempo (DRAW_FRAME) activa secuencialmente las 8 filas. Para eliminar el efecto *ghosting* (sombras), la rutina DISABLE_ALL_ROWS apaga temporalmente todas las filas (nivel ALTO) antes de cargar los nuevos datos de columna.

Algoritmo de Desplazamiento (Scrolling): La rutina ADVANCE_SCROLL_STEP lee la memoria Flash con el puntero Z (lpm). La subrutina SHIFT_LOOP desplaza 1 bit a la derecha el buffer en RAM (lsr) e inserta la nueva columna en el bit MSB (bit 7). Al detectar la marca 0xFF, se reinicia el puntero MSG_PTR.

Optimización del Hardware UART y Reasignación de Pin: La rutina PRINT_UART_MENU transmite el menú inicial a 9600 baudios. Tras verificar la finalización de la transmisión (TXC0), se desactiva el transmisor (TXEN0 en UCSR0B). La Columna 1 de la matriz se reasignó al pin analógico A5 (PC5) en lugar del pin PD1 (TX) para evitar interferencias de hardware con el chip convertidor USB-Serie del Arduino.

IV-D. Montaje Físico y Conexionado

Se realizó el armado del circuito sobre protoboard utilizando el microcontrolador ATmega328P en la placa Arduino Uno. En las Tablas II y III se detalla la correspondencia exacta entre los pines físicos de la matriz de LEDs 8 × 8 (1088AS), su función interna y los pines mapeados.

Cuadro II
PINES INFERIORES DE LA MATRIZ 1088AS (PINES 1 AL 8)

Pin 1088AS	Función	Arduino Uno	ATmega328P
Pin 1	Fila 5	D13	PB5
Pin 2	Fila 7	A1	PC1
Pin 3	Columna 2	D2	PD2
Pin 4	Columna 3	D3	PD3
Pin 5	Fila 8	A2	PC2
Pin 6	Columna 5	D5	PD5
Pin 7	Fila 6	A0	PC0
Pin 8	Fila 3	D11	PB3

Cuadro III
PINES SUPERIORES DE LA MATRIZ 1088AS (PINES 9 AL 16)

Pin 1088AS	Función	Arduino Uno	ATmega328P
Pin 9	Columna 1	A5	PC5
Pin 10	Fila 4	D12	PB4
Pin 11	Columna 6	D6	PD6
Pin 12	Fila 2	D10	PB2
Pin 13	Columna 4	D4	PD4
Pin 14	Columna 8	D8	PB0
Pin 15	Fila 1	D9	PB1
Pin 16	Columna 7	D7	PD7

En la Fig. 5 se observa la implementación física del circuito en protoboard.

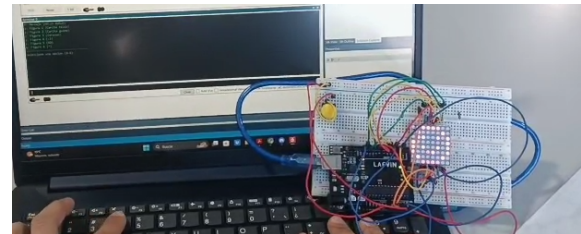


Figura 5. Montaje físico del circuito en protoboard.

IV-E. Simulación en PICSIMLab

Se realizó la simulación del sistema en la plataforma PICSIMLab configurada con el microcontrolador ATmega328P (Arduino Uno a 16 MHz). Esta etapa permitió validar el firmware en ensamblador antes del montaje físico.

A través del módulo de terminal virtual (VTerm) conectado vía UART a 9600 baudios, se verifica el despliegue del menú interactivo. El usuario puede enviar comandos ASCII para conmutar entre el texto dinámico “HELLO WORLD” (Modo 0) y los 6 patrones gráficos estáticos (Modos 1 al 6), así como regular la velocidad de desplazamiento mediante los caracteres + y –.

Cuadro IV
CONEXIONADO DE PINES EN SIMULACIÓN (PICSIMLAB)

Módulo	Pin Módulo	Pin ATmega328P / Arduino
Terminal Serial (VTerm)	RX	PD1 (Pin D1 - TX)
	TX	PD0 (Pin D0 - RX)
Módulo Matriz de LEDs	VCC	5V
	GND	GND
	DIN (Datos)	PB3 (Pin D11)
	CS (LOAD)	PB2 (Pin D10)
	CLK (Clock)	PB5 (Pin D13)

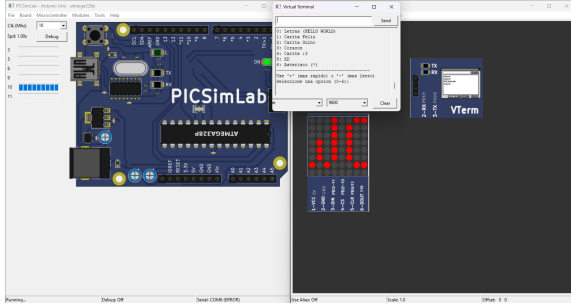


Figura 6. Simulación del sistema en PICSIMLAB con Terminal Virtual (VTerm) y Módulo Matriz.

IV-F. Conclusión (Parte A)

La implementación del control de la matriz de LEDs de 8×8 mediante lenguaje ensamblador en el microcontrolador ATmega328P cumplió satisfactoriamente con todos los objetivos planteados. Se logró validar la arquitectura del programa tanto de forma virtual en PICSIMLab como en el montaje físico sobre protoboard.

El desarrollo permitió integrar conceptos clave de microprocesadores:

- **Gestión eficiente de memoria:** Uso coordinado de la memoria Flash para el almacenamiento de tablas de patrones y texto, y la memoria SRAM para el manejo del buffer dinámico de desplazamiento.
- **Control de periféricos y tiempo real:** Implementación de barrido por multiplexación de filas a alta frecuencia con técnica de *blanking* para eliminar sombras (*ghosting*), junto con el manejo de la interfaz serie UART a 9600 baudios.
- **Flexibilidad de interfaz:** Interacción dual fluida por medio de pulsadores físicos con detección anti-rebote y comandos por terminal serie.

V. PROBLEMA B: GENERADOR DE FUNCIONES MEDIANTE DAC R-2R

V-A. Objetivo

Diseñar e implementar un programa en lenguaje ensamblador para el microcontrolador ATmega328P que controle un convertidor Digital-Analógico (DAC) con red de resistencias R-2R de 8 bits, permitiendo la generación sintética y conmutación de formas de onda precalculadas almacenadas en la memoria Flash. El sistema busca integrar el muestreo por tabla de búsqueda (*Look-Up Table* - LUT), el manejo del puntero

de programa *Z*, y el ajuste dinámico de la frecuencia de salida mediante la lectura de pulsadores de control, validando el resultado analógico en osciloscopio.

V-B. Metodología

La solución desarrollada en ensamblador AVR para el microcontrolador ATmega328P gestiona la conversión Digital-Analógica mediante la transferencia secuencial de muestras de 8 bits desde la memoria de programa hacia el puerto PORTD, con conmutación de señales en tiempo real y modulación de frecuencia mediante retardo por software.

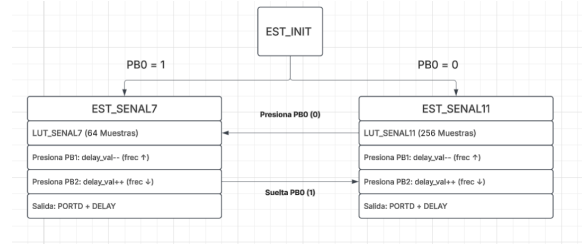


Figura 7. Diagrama de Flujo y Máquina de Estados Finitos (FSM).

V-B1. Máquina de Estados Finitos (FSM): Descripción de Estados:

- **Inicio (EST_INIT / Inicialización):** Configura el *Stack Pointer* (RAMEND), establece el puerto PORTD completo como salida digital (DDRD = 0xFF), inicializa los pines de entrada en PORTB habilitando las resistencias de *pull-up* internas y fija el valor inicial de retardo (*delay_val* = 20).
- **Estado 1 (EST_SENAL7 – Onda de 64 Muestras):** Carga el puntero *Z* (ZH:ZL) con la dirección de LUT_SENAL7 en memoria Flash. Mediante la instrucción *lpm*, lee e incrementa secuencialmente las 64 muestras enviándolas al puerto PORTD con una pausa definida por *delay_val*. Permanece en este estado mientras *PB0* = 1.
- **Estado 2 (EST_SENAL11 – Onda de 256 Muestras):** Se activa cuando el usuario presiona el botón selector (*PB0* = 0). Apunta el registro *Z* a la LUT_SENAL11 en Flash y transmite las 256 muestras de mayor resolución hacia el puerto PORTD aplicándole la subrutina de retardo.

Eventos de Transición:

- **Pulsador Selector de Señal (PB0):** Conmuta de forma directa entre EST_SENAL7 y EST_SENAL11 evaluando el nivel lógico del pin *PB0* (1 → Señal 7, 0 → Señal 11).
- **Pulsadores de Frecuencia (PB1 / PB2):** La detección de nivel bajo en *PB1* decrementa *delay_val* (hasta un límite inferior de 2), aumentando la frecuencia de salida. Presionar *PB2* incrementa *delay_val* (hasta un límite de 250), disminuyendo la frecuencia de la señal.

V-B2. *Implementación Técnica y Justificación de Diseño:*
Convertidor Digital a Analógico (DAC) R-2R: La red R-2R es una configuración pasiva que convierte el byte digital

enviado por PORTD (D_0-D_7) en una tensión analógica proporcional V_{out} :

$$V_{out} = V_{ref} \cdot \sum_{i=0}^7 \frac{D_i}{2^{8-i}} = V_{ref} \cdot \frac{\text{Dato Digital}}{256} \quad (1)$$

Con $V_{ref} = 5\text{ V}$, la resolución por escalón es de aproximadamente 19.53 mV.

Gestión de Memoria y Mapeo de Registros: El diseño utiliza tablas de búsqueda (*Look-Up Tables*) almacenadas en memoria Flash mediante la directiva `.db`. La lectura se realiza mediante direccionamiento indirecto con el puntero Z y la instrucción `lpm dato, Z+`.

Cuadro V
ASIGNACIÓN DE REGISTROS Y VARIABLES DEL SISTEMA

Reg. / Variable	Etiqueta	Función / Propósito
R16	TEMP	Registro auxiliar de configuración de E/S
R17	DATO	Muestra leída desde Flash a enviar a PORTD
R20	DELAY_VAL	Valor de retardo por software (Ajuste de Frecuencia)
ZH:ZL	Z_PTR	Registro Puntero de memoria Flash para lectura LUT

Control de Frecuencia por Software: La frecuencia de la señal generada se controla mediante la subrutina `DELAY`, cuyo tiempo de ejecución es proporcional al valor almacenado en `DELAY_VAL`. Al acortar el tiempo entre muestras, la señal completa su período más rápido, elevando la frecuencia.

V-C. Montaje Físico y Conexionado

Se realizó el armado del circuito utilizando el microcontrolador ATmega328P en la placa Arduino Uno conectado a una red atenuadora R-2R con valor nominal $R = 1\text{ k}\Omega$ y $2R = 2\text{ k}\Omega$. Debido a la indisponibilidad de resistencias comerciales de $2\text{ k}\Omega$ en el montaje físico, cada rama equivalente de $2R$ se implementó mediante la asociación en serie de dos resistencias de $1\text{ k}\Omega$ ($1\text{ k}\Omega + 1\text{ k}\Omega = 2\text{ k}\Omega$). En las Tablas I y II se detalla la correspondencia de pines.

Cuadro VI
ASIGNACIÓN DE PINES DE SALIDA HACIA EL DAC R-2R

Pin Arduino	Puerto ATmega	Tipo	Función / Módulo DAC
D0--D7	PD0--PD7	Salida	Bus de Datos de 8 bits (D_0-D_7) hacia R-2R

Cuadro VII
PINES DE ENTRADA E INTERFAZ DE CONTROL

Pin Arduino	Puerto ATmega	Tipo	Función / Dispositivo
D8	PB0	Entrada (<i>Pull-up</i>)	Botón Selector (1 → Señal 7, 0 → Señal 11)
D9	PB1	Entrada (<i>Pull-up</i>)	Pulsador Incrementar Frecuencia (Decrementar delay)
D10	PB2	Entrada (<i>Pull-up</i>)	Pulsador Disminuir Frecuencia (Incrementar delay)

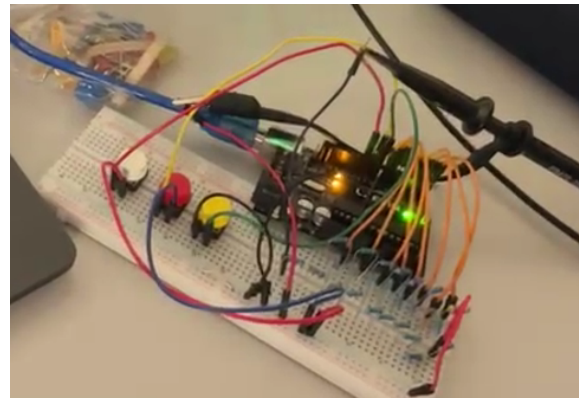


Figura 8. Montaje físico del circuito R-2R en protoboard y medición en osciloscopio.

V-D. Simulación en Proteus

Se realizó la simulación completa del sistema en el entorno Proteus configurado con el microcontrolador ATmega328P a 16 MHz y el esquema de la red R-2R. Esta etapa permitió validar la lógica del firmware en ensamblador antes del armado físico.

A través del osciloscopio virtual de Proteus conectado a la salida del DAC R-2R, se constató la correcta generación de ambas formas de onda: la `SENAL7` (perfil triangular de 64 muestras) y la `SENAL11` (onda compleja de 256 muestras). La interacción con los pulsadores virtuales en PB1 y PB2 confirmó la variación continua del período en la pantalla del osciloscopio sin distorsión en la amplitud.

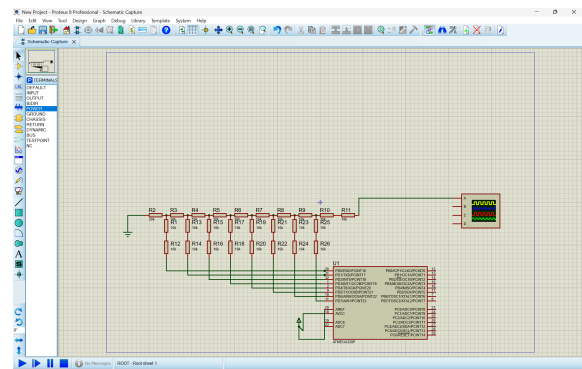


Figura 9. Simulación del circuito en Proteus con osciloscopio virtual.

V-E. Conclusión (Parte B)

La implementación del generador de funciones mediante el DAC R-2R de 8 bits y lenguaje ensamblador en el microcontrolador ATmega328P cumplió satisfactoriamente con todos los objetivos planteados. Se logró validar la arquitectura del programa tanto en la simulación realizada en Proteus como en el montaje físico sobre protoboard.

El desarrollo permitió integrar conceptos clave de microprocesadores:

- **Gestión eficiente de memoria:** Uso del puntero Z e instrucción `lpm` para extraer tablas de síntesis analógica desde la memoria Flash sin comprometer la memoria RAM.

- **Conversión D/A pasiva:** Generación de tensiones continuas a través de la red R-2R con resolución de 8 bits (19.53 mV por escalón), resolviendo la falta de componentes mediante la combinación en serie de resistencias de 1 k Ω .
- **Control dinámico de señal:** Selección instantánea de forma de onda y escalado directo de frecuencia mediante la lectura de pulsadores en bucle de control.

VI. EJERCICIO C: PLOTTER

VI-A. Objetivo

Diseñar e implementar el sistema de control digital para un plotter mediante el microcontrolador ATmega328P en lenguaje ensamblador, procesando órdenes de trazado de figuras geométricas recibidas vía comunicación serie (UART) para generar las secuencias de movimiento coordinado en los ejes XY y el accionamiento sincrónico del solenoide del lápiz.

VI-B. Metodología

Para este ejercicio se desarrolló un sistema de control para un plotter mediante el microcontrolador ATmega328P, programado en lenguaje ensamblador. El microcontrolador se comunica con el plotter a través de las entradas del PLC, utilizando señales digitales para controlar el movimiento de los ejes y el accionamiento del lápiz.

El sistema permite seleccionar mediante comunicación UART diferentes figuras para ser dibujadas. Las opciones implementadas corresponden al triángulo, círculo, pentagrama, una figura personalizada, en nuestro caso las letras UTEC, el Pokémon asignado y una opción para ejecutar todas las figuras de forma consecutiva.

La comunicación serial fue configurada a una velocidad de 9600 baudios, con una configuración de 8 bits de datos, sin paridad y un bit de parada. El programa permanece esperando un carácter recibido por UART y, dependiendo del carácter ingresado, ejecuta la rutina de dibujo correspondiente.

Para controlar los movimientos del plotter se utilizaron las salidas digitales del puerto D del ATmega328P. Cada movimiento se implementó mediante una rutina independiente para desplazarse hacia arriba, abajo, izquierda o derecha. También se implementaron rutinas para realizar movimientos diagonales, permitiendo generar figuras más complejas.

El accionamiento del lápiz se realiza mediante dos señales correspondientes a subir y bajar el solenoide. Antes de comenzar cada figura, el programa realiza un desplazamiento de seguridad denominado *OFFSET_MARGIN*, que mueve el cabezal hacia una posición dentro del área de dibujo manteniendo el lápiz levantado.

Además, se implementaron retardos para controlar el tiempo durante el cual se mantienen activadas las señales de movimiento. Estos retardos permiten que el plotter tenga tiempo suficiente para realizar cada desplazamiento y estabilizarse antes de ejecutar el siguiente movimiento.

VI-C. Diagrama maquina de estados

El funcionamiento general del programa puede representarse mediante una máquina de estados. Luego del reinicio, el sistema configura el Stack Pointer, los pines del puerto D y la comunicación UART. Posteriormente, permanece en un estado de espera hasta recibir un comando mediante el puerto serial.

Según el carácter recibido, el programa selecciona la rutina correspondiente y ejecuta la figura solicitada. Una vez finalizado el dibujo, el lápiz es levantado y el sistema regresa nuevamente al estado de espera para recibir una nueva orden.

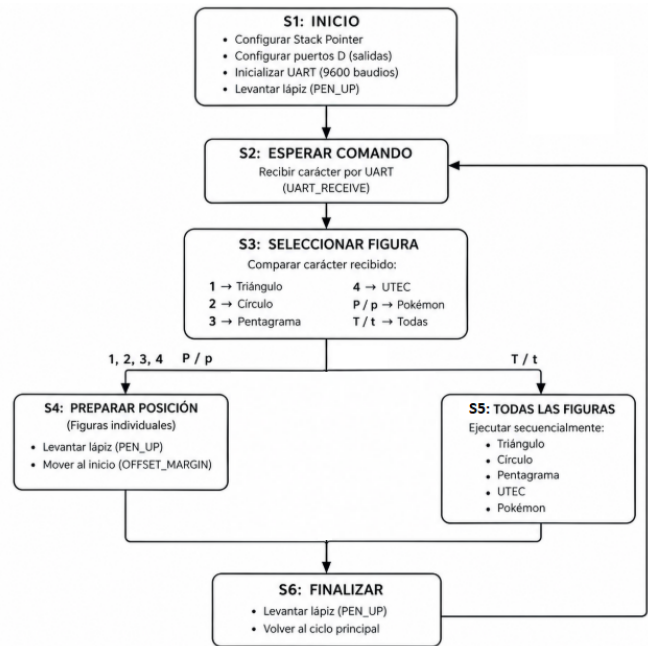


Figura 10. Diagrama de funcionamiento del sistema del Plotter.

VI-D. Descripción de los estados

El funcionamiento del sistema puede dividirse en diferentes estados de operación, relacionados con las principales etapas ejecutadas por el programa.

VI-D1. S1 – Inicialización: Este es el estado inicial del sistema. Al comenzar la ejecución se configura el Stack Pointer utilizando la dirección final de la memoria RAM del ATmega328P.

También se configura el puerto D como salida para controlar los movimientos del plotter y los solenoides. Finalmente, se inicializa la comunicación UART a 9600 baudios.

Una vez completada la configuración, el programa levanta el lápiz mediante la rutina *PEN_UP* y pasa al estado de espera.

VI-D2. S2 – Espera de comando UART: En este estado el microcontrolador permanece esperando la recepción de un carácter mediante el puerto serial.

La rutina *UART_RECEIVE* verifica continuamente el bit *RXC0* del registro *UCSR0A*. Cuando se recibe un dato, este es leído desde el registro *UDR0* y almacenado en el registro *r16*.

Luego, el programa compara el carácter recibido con las diferentes opciones disponibles para determinar qué figura debe ejecutarse.

VI-D3. S3 – Selección de figura: Una vez recibido el comando, el programa compara el valor almacenado en `r16` con los caracteres correspondientes a cada figura.

El carácter '1' selecciona el triángulo, '2' selecciona el círculo, '3' selecciona el pentagrama y '4' selecciona la figura personalizada UTEC.

También se acepta la letra 'P' o 'p' para seleccionar el Pokémon asignado, mientras que la letra 'T' o 't' permite ejecutar todas las figuras.

Si el carácter recibido no coincide con ninguna de estas opciones, el programa regresa al ciclo principal y continúa esperando un nuevo comando.

VI-D4. S4 – Preparación de la posición y ejecución del dibujo: Antes de ejecutar cualquiera de las figuras, se llama a la rutina `OFFSET_MARGIN`.

Esta rutina mantiene el lápiz levantado y desplaza el cabezal hacia una posición segura dentro del área de dibujo. Primero se realiza un desplazamiento hacia la izquierda y posteriormente un desplazamiento hacia abajo.

De esta forma se evita comenzar el dibujo directamente desde la posición inicial del plotter y se establece un margen de seguridad antes de realizar la figura.

En este estado se ejecuta la rutina correspondiente a la figura seleccionada.

Durante el dibujo, el programa utiliza las rutinas de movimiento para generar cada uno de los trazos. El valor cargado en el registro `r17` determina la cantidad de tiempo durante la cual se mantiene activo cada movimiento.

Dependiendo de la figura, pueden utilizarse movimientos horizontales, verticales o diagonales. Durante los trazos se utiliza la rutina `PEN_DOWN` para bajar el lápiz y comenzar a dibujar.

VI-D5. S5 – Ejecución de todas las figuras: Cuando se recibe el comando 'T' o 't', se ejecuta la rutina `DRAW_ALL`.

Esta rutina ejecuta consecutivamente las diferentes figuras implementadas. Entre algunas de ellas se realizan desplazamientos adicionales para separar los dibujos y evitar que se superpongan.

La secuencia incluye el triángulo, círculo, pentagrama, figura UTEC y Pokémon asignado.

VI-D6. S6 – finalización del programa: en este apartado se levanta el lápiz y vuelve a el ciclo principal (S2)

VI-E. Asignación de pines

Para controlar el plotter se utilizaron los pines digitales D2 a D7 del microcontrolador. Las señales fueron asignadas de la siguiente manera:

- D2: bajar el solenoide del lápiz.
- D3: subir el solenoide del lápiz.
- D4: movimiento hacia abajo.
- D5: movimiento hacia arriba.

- D6: movimiento hacia la derecha.
- D7: movimiento hacia la izquierda.

VI-F. Control de los movimientos

Los movimientos del plotter fueron implementados mediante diferentes subrutinas.

Para los desplazamientos principales se utilizaron las rutinas:

- `MOVE_DOWN`: movimiento hacia abajo.
- `MOVE_UP`: movimiento hacia arriba.
- `MOVE_LEFT`: movimiento hacia la izquierda.
- `MOVE_RIGHT`: movimiento hacia la derecha.

También se implementaron movimientos diagonales mediante las rutinas:

- `MOVE_DN_LT`: abajo e izquierda.
- `MOVE_DN_RT`: abajo y derecha.
- `MOVE_UP_LT`: arriba e izquierda.
- `MOVE_UP_RT`: arriba y derecha.

Cada una de estas rutinas activa la salida correspondiente del puerto D durante un tiempo determinado y luego desactiva la señal. Posteriormente se ejecuta un pequeño retardo de estabilización antes de continuar con el siguiente movimiento.

VI-G. Control del lápiz

El control del lápiz se realiza mediante dos señales digitales conectadas a los solenoides del plotter.

La rutina `PEN_DOWN` activa la señal correspondiente a bajar el lápiz y desactiva la señal de subida. De esta manera, el lápiz queda en contacto con la superficie y los siguientes movimientos generan el trazado.

Por otro lado, la rutina `PEN_UP` desactiva la señal de bajada y activa la señal de subida, permitiendo desplazar el cabezal sin realizar un trazo.

Luego de cada cambio de posición del lápiz se ejecuta la rutina `DELAY_SOLENOID`, que introduce un retardo para permitir que el mecanismo termine de realizar el movimiento.

VI-H. Control de tiempos

Los tiempos de movimiento se controlan utilizando el registro `r17`. Antes de llamar a una rutina de movimiento se carga en este registro un valor que determina la duración del desplazamiento.

La rutina `WAIT_R17_50MS` ejecuta repetidamente un retardo de aproximadamente 50 ms, decrementando el valor almacenado en `r17` hasta llegar a cero.

Por ejemplo, para realizar un movimiento con un valor de:

```
ldi r17, 5
rcall MOVE_LEFT
```

la rutina mantiene activado el movimiento hacia la izquierda durante cinco intervalos de aproximadamente 50 ms.

De esta manera, el tiempo de movimiento puede modificarse fácilmente cambiando el valor cargado en `r17`.

VI-I. Figuras implementadas

El programa permite seleccionar diferentes figuras mediante los comandos recibidos por UART.

VI-I1. Triángulo: La primera figura se ejecuta mediante la rutina `DRAW_TRIANGLE`. Para realizarla se baja el lápiz y se realizan tres movimientos principales: una diagonal hacia abajo y a la izquierda, un movimiento hacia la derecha para formar la base y finalmente una diagonal hacia arriba y a la izquierda para cerrar la figura.

VI-I2. Círculo: La segunda figura se implementó mediante la rutina `DRAW_CIRCLE`. Debido a las características del sistema de movimiento, el círculo se aproxima mediante una secuencia de pequeños desplazamientos horizontales, verticales y diagonales.

De esta manera se obtiene una figura poligonal que aproxima visualmente una circunferencia.

VI-I3. Pentagrama: La tercera figura corresponde a un pentagrama y se encuentra implementada mediante la rutina `DRAW_PENTAGRAM`.

Para generar la figura se utilizan múltiples movimientos sucesivos del cabezal, combinando desplazamientos horizontales y verticales para construir los diferentes trazos que forman la estrella.

VI-I4. Figura UTEC: La cuarta figura corresponde a una representación de la palabra UTEC.

La rutina `DRAW_UTEC` genera cada letra mediante segmentos rectos. Para separar las letras se realizan desplazamientos adicionales con el lápiz levantado, evitando generar trazos entre una letra y otra.

VI-I5. Pokémon: También se implementó una rutina denominada `DRAW_POKEMON`, correspondiente al Pokémon asignado al grupo.

Debido a la complejidad de la figura, esta rutina está compuesta por una gran cantidad de pequeños desplazamientos. Estos movimientos permiten aproximar el contorno y los detalles de la figura utilizando las capacidades de movimiento del plotter.

VI-I6. Todas las figuras: Finalmente, la rutina `DRAW_ALL` permite ejecutar las figuras de forma consecutiva.

La secuencia comienza con el triángulo, seguido del círculo y el pentagrama. Posteriormente se dibuja la figura UTEC y finalmente el Pokémon asignado.

Entre las diferentes figuras se realizan desplazamientos adicionales con el lápiz levantado para generar una separación entre los dibujos.

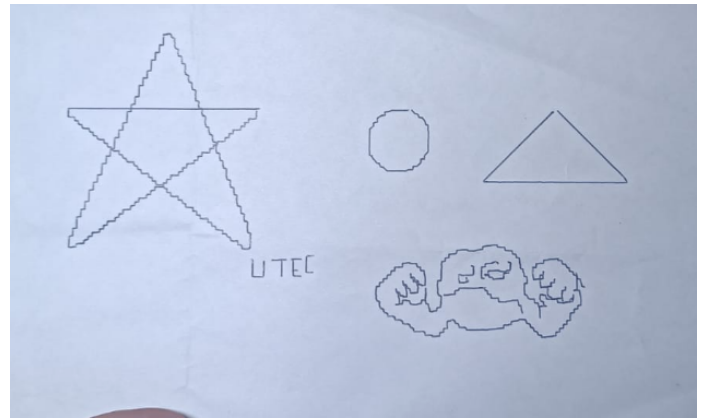


Figura 11. Todas las figuras implementadas.

VI-J. Resultados

Mediante la implementación realizada se logró controlar el plotter utilizando el ATmega328P y comunicación serial.

El sistema permite seleccionar diferentes figuras mediante comandos enviados por UART y ejecutar automáticamente las secuencias de movimiento correspondientes. Los movimientos se realizan mediante señales digitales enviadas al PLC, mientras que el lápiz es controlado mediante las rutinas de subida y bajada del solenoide.

La utilización de rutinas independientes para cada movimiento permitió simplificar la programación de las figuras, ya que cada dibujo puede construirse a partir de una secuencia de desplazamientos horizontales, verticales y diagonales.

También se incorporó un desplazamiento inicial de seguridad para evitar comenzar el dibujo directamente en el extremo de la zona de trabajo.

VI-K. Conclusiones

En este ejercicio se logró implementar el control de un plotter mediante un microcontrolador ATmega328P programado en lenguaje ensamblador.

La comunicación UART permitió seleccionar las figuras mediante comandos enviados desde el puerto serial, mientras que las diferentes rutinas de movimiento permitieron transformar cada figura en una secuencia de desplazamientos del cabezal.

El uso de retardos fue necesario para controlar adecuadamente los tiempos de activación de las señales y permitir que el mecanismo del plotter pudiera realizar cada movimiento antes de continuar con el siguiente.

Finalmente, la implementación permitió integrar diferentes conceptos trabajados durante el laboratorio, como configuración de puertos, comunicación UART, subrutinas, control de tiempos y manejo de registros.

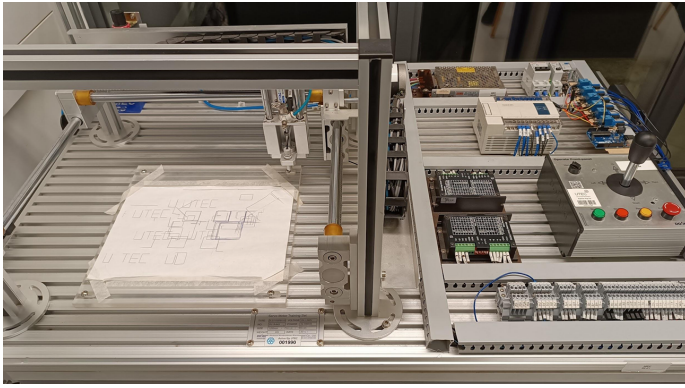


Figura 12. Plotter.

VII. EJERCICIO D: COMUNICACIÓN USART ENTRE EMISOR Y RECEPTOR

VII-A. Objetivo

Diseñar e implementar un enlace de comunicación serie asincrónica punto a punto (USART) entre dos microcontroladores ATmega328P programados en ensamblador, permitiendo la lectura y transmisión de combinaciones binarias de 3 bits desde pulsadores en el nodo emisor hacia el nodo receptor para la decodificación y activación de un banco de 8 LEDs.

VII-B. Metodología

Para este ejercicio se desarrolló un sistema de comunicación entre dos microcontroladores ATmega328P mediante el protocolo USART. Uno de los microcontroladores funciona como emisor y el otro como receptor.

El microcontrolador emisor dispone de tres pulsadores conectados a los pines PD2, PD3 y PD4. Cada pulsador representa un bit, permitiendo generar diferentes combinaciones binarias de tres bits y obtener valores comprendidos entre 0 y 7.

El valor generado a partir de los pulsadores es transmitido mediante la interfaz USART hacia el segundo microcontrolador. El receptor interpreta el dato recibido y utiliza su valor para determinar cuál de los ocho LEDs debe encender.

Antes de activar el LED correspondiente al nuevo dato recibido, el programa receptor apaga todas las salidas. De esta manera, se evita que permanezca encendido el LED asociado a una selección anterior.

La comunicación USART se configuró utilizando una velocidad de 9600 bps, 8 bits de datos, sin paridad y un bit de parada.

El funcionamiento general del sistema puede resumirse en las siguientes etapas:

1. Inicialización del microcontrolador emisor.
2. Configuración de los tres pulsadores como entradas.
3. Lectura de los pulsadores.
4. Formación de un valor binario de tres bits.
5. Transmisión del valor mediante USART.
6. Recepción del dato en el segundo microcontrolador.

7. Validación del valor recibido.
8. Apagado de los LEDs.
9. Selección y activación del LED correspondiente.
10. Espera de un nuevo dato.

VII-C. Diagrama de máquina de estados del Emisor

El funcionamiento del microcontrolador emisor puede representarse mediante una secuencia de estados que comienza con la inicialización del sistema y continúa con la lectura de los pulsadores y la transmisión del dato.

Luego de transmitir el valor correspondiente a la combinación seleccionada, el programa realiza un retardo y vuelve nuevamente a la lectura de los pulsadores.

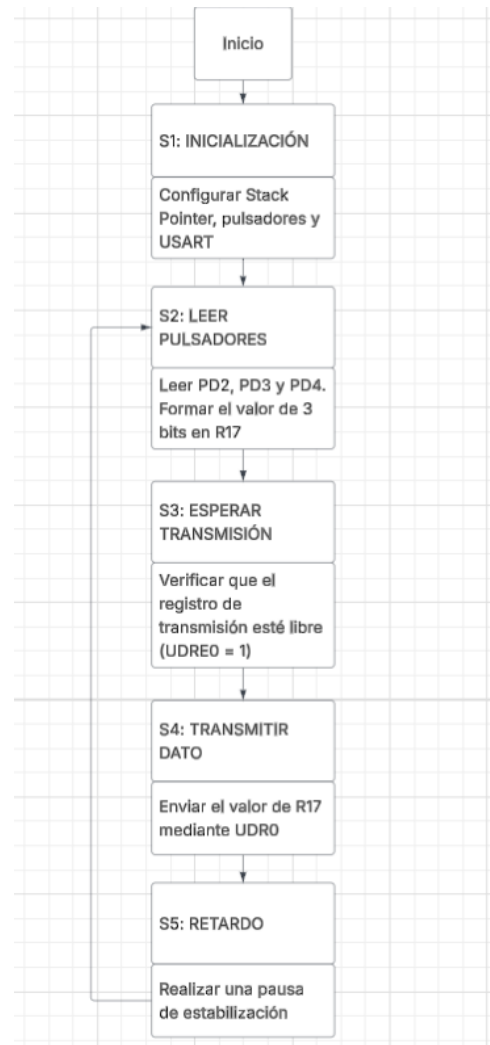


Figura 13. Diagrama de estados correspondiente al microcontrolador emisor.

VII-D. Descripción de los estados del Emisor

VII-D1. S1 – Inicialización: Este corresponde al estado inicial del sistema. Durante esta etapa se configura el Stack Pointer del ATmega328P y se establecen los pines utilizados por los pulsadores como entradas digitales.

También se inicializa la comunicación USART con una configuración de 9600 bps, 8 bits de datos, sin paridad y un bit de parada.

Una vez finalizada la configuración, el programa continúa con la lectura de los pulsadores.

VII-D2. S2 – Lectura de pulsadores: En este estado se leen individualmente los tres pulsadores conectados a PD2, PD3 y PD4.

Debido a la utilización de resistencias Pull-Up, los pines presentan un nivel lógico alto cuando los pulsadores no están presionados y un nivel lógico bajo cuando se presionan.

El programa comienza con el registro R17 en cero y posteriormente modifica los bits correspondientes según el estado de cada pulsador.

La correspondencia utilizada es:

- PD2 → bit 0.
- PD3 → bit 1.
- PD4 → bit 2.

VII-D3. S3 – Espera de transmisión: Antes de transmitir el dato, el programa verifica que el registro de transmisión de la USART se encuentre disponible.

Para esto se consulta el bit UDRE0 del registro UCSRA. Cuando UDRE0 se encuentra en uno, significa que el registro de datos de transmisión está disponible para recibir un nuevo valor.

Mientras el registro no se encuentre disponible, el programa permanece esperando.

VII-D4. S4 – Transmisión del dato: Una vez que la USART se encuentra preparada, el contenido del registro R17 se copia al registro UDR0.

La operación utilizada es:

```
sts UDR0, r17
```

El valor almacenado en R17 es enviado entonces mediante la línea de transmisión USART hacia el segundo microcontrolador.

VII-D5. S5 – Retardo: Luego de realizar la transmisión se ejecuta una rutina de retardo mediante la subrutina DELAY.

Este retardo permite establecer un intervalo entre transmisiones consecutivas. Una vez finalizado, el programa regresa nuevamente a la lectura de los pulsadores.

De esta manera, el sistema permanece funcionando continuamente y puede transmitir nuevos valores cuando cambia la combinación seleccionada.

VII-E. Diagrama de máquina de estados del Receptor

El microcontrolador receptor permanece esperando continuamente la llegada de datos mediante USART. Cuando se recibe un valor, este es leído, validado y utilizado para seleccionar una de las ocho salidas disponibles.



Figura 14. Diagrama de estados correspondiente al microcontrolador receptor.

VII-F. Descripción de los estados del Receptor

VII-F1. S1 – Inicialización: Durante este estado se configura el Stack Pointer y se establecen como salidas los pines utilizados para controlar los ocho LEDs.

Los primeros seis LEDs se encuentran conectados a los pines PD2–PD7, mientras que los dos LEDs restantes se conectan a PB0 y PB1.

También se configura el módulo USART para recibir datos a 9600 bps, utilizando 8 bits de datos, sin paridad y un bit de parada.

Una vez finalizada la inicialización, el programa pasa al estado de espera.

VII-F2. S2 – Espera de dato: El receptor permanece en este estado hasta detectar la recepción de un nuevo dato mediante USART.

Para determinar si existe un dato disponible se verifica el bit RXC0 del registro UCSRA.

Mientras `RXC0` permanezca en cero, el programa continúa esperando. Cuando este bit pasa a uno, significa que existe un dato recibido disponible en el registro `UDR0`.

VII-F3. S3 – Lectura del dato: Cuando se detecta la recepción de un dato, el programa lee el registro `UDR0` y almacena su contenido en el registro `R16`.

La operación utilizada es:

```
lds r16, UDR0
```

De esta manera, el valor recibido queda disponible en `R16` para ser procesado por el programa.

VII-F4. S4 – Validación del dato: Luego de recibir el valor, el programa verifica que este se encuentre dentro del rango esperado.

El sistema trabaja con tres bits, por lo que los únicos valores válidos son los comprendidos entre 0 y 7.

Para realizar esta comprobación se utiliza la comparación:

```
cpi r16, 8
brsh MAIN_LOOP
```

Si el valor recibido es mayor o igual a 8, se considera inválido y el programa regresa al ciclo principal para esperar un nuevo dato.

Por lo tanto, el rango aceptado por el receptor es:

$$0 \leq R16 \leq 7 \quad (2)$$

VII-F5. S5 – Apagado de LEDs: Cuando el valor recibido es válido, el programa ejecuta la rutina `APAGARLEDS`. Esta rutina desactiva todas las salidas utilizadas para controlar los LEDs antes de seleccionar la nueva salida.

Esta operación permite garantizar que solamente permanezca activo el LED correspondiente al último valor recibido.

VII-F6. S6 – Selección del LED: Luego de apagar todas las salidas, el programa compara el valor almacenado en `R16` con los diferentes valores posibles.

La correspondencia entre el dato recibido y el LED seleccionado es:

- 0 → LED 0.
- 1 → LED 1.
- 2 → LED 2.
- 3 → LED 3.
- 4 → LED 4.
- 5 → LED 5.
- 6 → LED 6.
- 7 → LED 7.

Cada comparación permite determinar qué rutina debe ejecutarse para activar la salida correspondiente.

VII-F7. S7 – Activación del LED: Finalmente, el programa activa el pin asociado al valor recibido.

La relación entre los LEDs y los pines del microcontrolador es:

- LED 0 → PD2.
- LED 1 → PD3.
- LED 2 → PD4.
- LED 3 → PD5.
- LED 4 → PD6.
- LED 5 → PD7.
- LED 6 → PB0.
- LED 7 → PB1.

Una vez activada la salida correspondiente, el programa regresa al estado de espera para recibir el siguiente dato.

VII-G. Explicación técnica de la implementación en Assembler

La programación de ambos microcontroladores se realizó utilizando lenguaje ensamblador para el ATmega328P. Esta implementación permite trabajar directamente con los registros internos del microcontrolador y controlar tanto las entradas de los pulsadores como las salidas de los LEDs y el módulo USART.

El programa se encuentra dividido en diferentes rutinas, permitiendo separar las funciones de inicialización, lectura de entradas, transmisión, recepción, validación y control de las salidas.

VII-G1. Uso de registros: En el microcontrolador emisor se utiliza principalmente el registro `R17` para almacenar el valor binario generado mediante los tres pulsadores.

Cada bit del registro representa uno de los pulsadores:

- Bit 0: pulsador conectado a PD2.
- Bit 1: pulsador conectado a PD3.
- Bit 2: pulsador conectado a PD4.

En el receptor, el dato recibido mediante USART se almacena inicialmente en el registro `R16`. Este registro es utilizado posteriormente para realizar las comparaciones que permiten determinar cuál de los ocho LEDs debe activarse.

VII-G2. Configuración de USART: La comunicación USART constituye el mecanismo de intercambio de información entre ambos microcontroladores.

La configuración utilizada es:

- Velocidad: 9600 bps.
- Bits de datos: 8.
- Paridad: ninguna.
- Bits de parada: 1.

En el emisor se habilita el transmisor mediante el bit `TXEN0`, mientras que en el receptor se habilita el módulo de recepción mediante el bit `RXEN0`.

La velocidad de transmisión se establece mediante los registros `UBRR0H` y `UBRR0L`. Para una frecuencia de reloj de 16 MHz y una velocidad de 9600 bps se utiliza el valor correspondiente configurado en dichos registros.

VII-G3. Lógica de selección: La selección del LED se realiza mediante comparaciones sucesivas entre el valor recibido y los valores comprendidos entre 0 y 7.

Esta implementación permite asociar directamente cada combinación binaria de los pulsadores con una salida específica del microcontrolador receptor.

Por ejemplo, si los pulsadores generan el valor binario:

011

el valor decimal correspondiente es 3, por lo que el receptor activa el LED asociado a la selección número 3.

VII-H. Comunicación USART

La comunicación entre ambos microcontroladores se realiza mediante el módulo USART integrado en el ATmega328P.

Para que la comunicación funcione correctamente, ambos dispositivos utilizan la misma configuración de velocidad y formato de trama.

La configuración utilizada es:

- 9600 bps.
- 8 bits de datos.
- Sin paridad.
- 1 bit de parada.

El microcontrolador emisor utiliza la línea de transmisión para enviar el dato, mientras que el receptor utiliza la línea de recepción para obtenerlo.

Además, ambos microcontroladores deben compartir una referencia común de GND para garantizar niveles lógicos compatibles durante la comunicación.

VII-I. Funcionamiento de los pulsadores

Los tres pulsadores permiten generar las ocho combinaciones posibles de un número binario de tres bits.

La correspondencia entre cada entrada y el bit generado es:

$$PD4 = bit_2, \quad PD3 = bit_1, \quad PD2 = bit_0 \quad (3)$$

Las combinaciones posibles son:

- 000 → 0.
- 001 → 1.
- 010 → 2.
- 011 → 3.
- 100 → 4.
- 101 → 5.
- 110 → 6.
- 111 → 7.

El valor generado es almacenado en R17 y posteriormente transmitido mediante USART al segundo microcontrolador.

VII-J. Armado físico

Para la implementación física se utilizaron dos microcontroladores ATmega328P. Uno de ellos corresponde al emisor y el otro al receptor.

En el microcontrolador emisor se conectaron tres pulsadores a los pines PD2, PD3 y PD4. Estos pines utilizan las resistencias Pull-Up internas del microcontrolador.

En el receptor se conectaron ocho LEDs a los pines PD2–PD7 y PB0–PB1.

Los dos microcontroladores comparten la referencia de GND y se conectan mediante las líneas correspondientes de comunicación USART, permitiendo que el dato generado por el emisor sea recibido por el segundo microcontrolador.

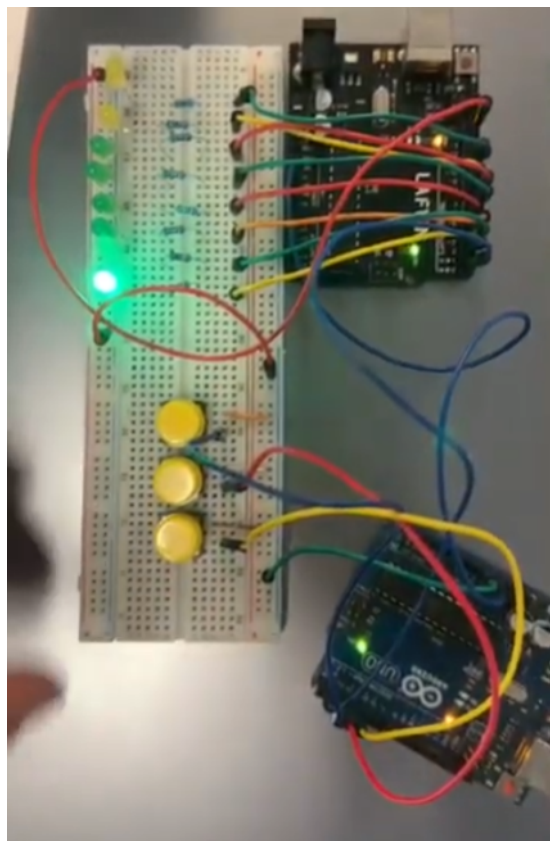


Figura 15. Armado físico correspondiente al ejercicio de comunicación USART.

VII-K. Resultados

Durante la ejecución del programa se comprobó que los tres pulsadores del microcontrolador emisor permiten generar las diferentes combinaciones binarias de tres bits.

El valor generado es transmitido mediante USART hacia el segundo microcontrolador, donde es recibido y procesado.

A partir del dato recibido, el receptor identifica el valor correspondiente y activa el LED asociado. De esta manera, las ocho combinaciones posibles permiten seleccionar las ocho salidas disponibles.

El funcionamiento implementado permite comprobar la comunicación entre ambos ATmega328P y la correcta utilización del dato transmitido para controlar las salidas del microcontrolador receptor.

VII-L. Conclusiones

En este ejercicio se implementó una comunicación USART entre dos microcontroladores ATmega328P programados en

lenguaje ensamblador.

El microcontrolador emisor permite generar valores de tres bits mediante tres pulsadores, obteniendo valores comprendidos entre 0 y 7. Estos valores son transmitidos mediante USART hacia el segundo microcontrolador.

El receptor interpreta el dato recibido y activa el LED correspondiente a la selección realizada.

La implementación permitió integrar diferentes recursos del ATmega328P, incluyendo la configuración de entradas y salidas digitales, utilización de resistencias Pull-Up, manejo de registros, comunicación USART y procesamiento de datos recibidos.

Además, el ejercicio permitió comprobar la utilización de una interfaz serial para transferir información entre dos microcontroladores y utilizar dicha información para controlar un conjunto de salidas digitales.

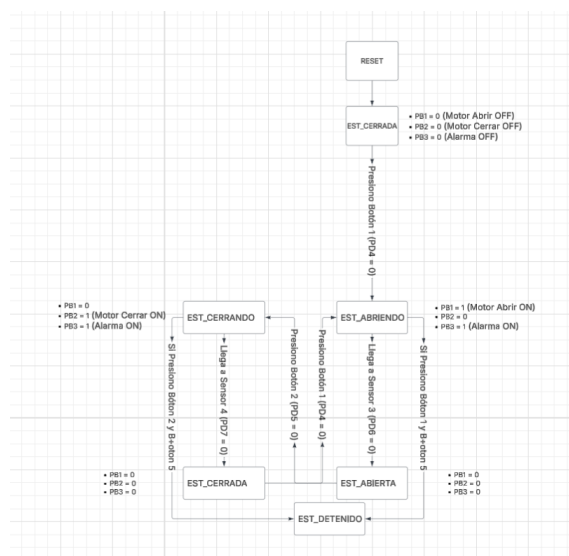


Figura 16. Diagrama de estados y flujo para la automatización de la puerta de garaje.

VIII. PROBLEMA E: AUTOMATIZACIÓN DE PUERTA DE GARAJE

VIII-A. Objetivo

Diseñar e implementar un sistema de control automatizado en lenguaje ensamblador para una puerta de garaje utilizando el microcontrolador ATmega328P. El sistema debe gestionar el sentido de giro de un motor (apertura/cierre), activar una alarma sonora/luminosa durante el movimiento, procesar sensores de posición de final de carrera (S_1 y S_2), responder a un control remoto por pulsadores y garantizar la seguridad mediante un sensor de obstáculos (S_3) asociado a una interrupción externa no vectorial de tipo *Pin Change Interrupt* (PCINT0). Asimismo, se debe reportar en tiempo real el estado del sistema a través del puerto serie USART a 9600 baudios.

VIII-B. Metodología y Máquina de Estados Finitos (FSM)

El control del sistema se estructuró mediante una Máquina de Estados Finitos (FSM) secuencial de 5 estados, garantizando que el motor y la alarma respondan únicamente a transiciones válidas de los sensores y comandos del usuario.

VIII-B1. Descripción de Estados y Transiciones:

- **Estado 0 (EST_CERRADA):** Estado de reposo inicial. El motor y la alarma permanecen desactivados. El sistema evalúa el pulsador de apertura en PD4.
- **Estado 1 (EST_ABIERTO):** Se activa la salida del motor de apertura (PB1) y la alarma (PB3). Transiciona a EST_ABIERTA cuando el sensor de final de carrera superior S_1 (PD6) pasa a nivel bajo (0).
- **Estado 2 (EST_ABIERTA):** La puerta alcanza su apertura máxima. Motores y alarma se apagan. El sistema espera la orden de cierre mediante el pulsador en PD5.
- **Estado 3 (EST_CERRANDO):** Se activa la salida del motor de cierre (PB2) y la alarma (PB3). Transiciona a EST_CERRADA al activarse el sensor de final de carrera inferior S_2 (PD7).
- **Estado 4 / Reversa de Seguridad (EST_DETENIDO):** Ante la detección de un obstáculo (S_3) en movimiento, la interrupción ISR_PCINT0 interrumpe el flujo normal, detiene o invierte el sentido de marcha del motor por seguridad y transmite el evento correspondiente vía USART.

VIII-C. Implementación Técnica y Mapeo de E/S

Asignación de Pines del Microcontrolador: Todas las entradas digitales fueron configuradas con resistencias *pull-up* internas habilitadas, operando con lógica invertida (activas en nivel bajo 0).

Cuadro VIII
ASIGNACIÓN DE PINES DE ENTRADA Y SALIDA (ATMEGA328P)

Pin Arduino	Puerto ATmega	Tipo	Función / Dispositivo
D8	PB0	Entrada (PCINT0)	Sensor Obstáculo (S_3 Barrera IR)
D9	PB1	Salida	Motor M1 (Orden Abrir)
D10	PB2	Salida	Motor M1 (Orden Cerrar)
D11	PB3	Salida	Alarma Acústica / Luminosa (A_1)
D4	PD4	Entrada	Control Remoto (Pulsador Abrir)
D5	PD5	Entrada	Control Remoto (Pulsador Cerrar)
D6	PD6	Entrada	Sensor S_1 (Puerta Abierta)
D7	PD7	Entrada	Sensor S_2 (Puerta Cerrada)
D0	PD0	Entrada	USART RX
D1	PD1	Salida	USART TX

Mecanismo de Interrupción de Seguridad (PCINT0):

Para cumplir con la restricción de no utilizar las interrupciones externas INT0 ni INT1, se configuró la interrupción por cambio de pin PCINT0 en el pin PB0 mediante el registro PCMSK0 y la habilitación de PCIE0 en PCICR. Al detectar un flanco descendente causado por el sensor de presencia S_3 , la rutina de servicio a la interrupción (ISR_PCINT0) evalúa el sentido actual: si la puerta se está cerrando, invierte inmediatamente el sentido hacia apertura para evitar aplastamientos; si se está abriendo, la conmuta a cierre y envía la advertencia por serie.

Comunicación Serie USART: El módulo USART fue configurado a 9600 baudios ($UBRR0 = 103$ a $f_{clk} = 16$ MHz) en formato de 8 bits de datos sin paridad y 1 bit de parada. Las cadenas de texto con los mensajes de estado se almacenan en la memoria Flash en formato .db de bytes nulos (NULL-terminated) y son leídas con direccionamiento indirecto mediante el registro puntero Z e instrucciones lpm.

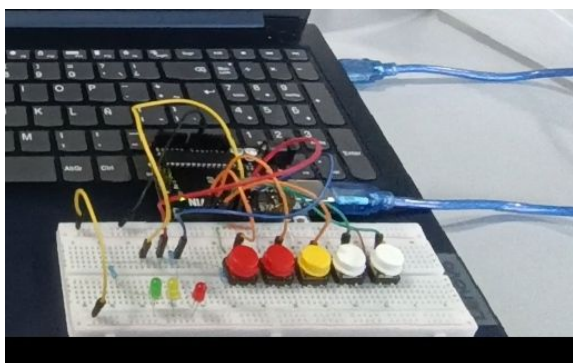


Figura 17. Esquema físico del sistema de automatización del garaje.

VIII-D. Simulación en PICSIMLAB

Se realizó la simulación del sistema de automatización de la puerta de garaje en la plataforma PICSIMLAB configurada con el microcontrolador ATmega328P (placa Arduino Uno a 16 MHz). Esta etapa permitió validar de forma virtual la lógica de la FSM, la atención inmediata de la interrupción por cambio de pin (PCINT0) y la transmisión de eventos a través de la interfaz USART antes del ensamblaje físico.

Para la simulación se utilizó el módulo de componentes virtuales (Spare Parts), integrando un bloque de pulsadores para simular el control remoto y los sensores de posición, un bloque de indicadores LED para representar los estados de los motores y la alarma, y el módulo de terminal virtual (VTerm)

conectado a los pines de transmisión serie (PD0 y PD1) a 9600 baudios.

Cuadro IX
MAPEO DE PERIFÉRICOS EN SIMULACIÓN (PICSIMLAB)

Componente Virtual	Pin ATmega	Pin Arduino	Señal / Función
Pulsador 1	PD4	D4	Control Remoto (Abrir)
Pulsador 2	PD5	D5	Control Remoto (Cerrar)
Pulsador 3	PD6	D6	Sensor S_1 (Puerta Abierta)
Pulsador 4	PD7	D7	Sensor S_2 (Puerta Cerrada)
Pulsador 5	PB0	D8	Sensor S_3 (Obstáculo / PCINT0)
LED Verde	PB1	D9	Motor M1 (Orden Abrir)
LED Amarillo	PB2	D10	Motor M1 (Orden Cerrar)
LED Rojo	PB3	D11	Alarma Acústica / Luminosa (A_1)
VTerm RX	PD1	TX / D1	Salida Serie USART
VTerm TX	PD0	RX / D0	Entrada Serie USART

Como se observa en la Figura 18, al accionar el pulsador de apertura en PD4, el sistema transiciona correctamente al estado EST_ABRIENDO, encendiendo la orden del motor de apertura (PB1) junto con la alarma (PB3) y enviando de forma inmediata la cadena de caracteres "Puerta abriendo" a la terminal serie VTerm.

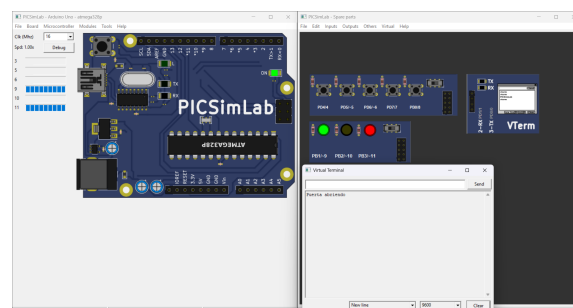


Figura 18. Simulación en PICSIMLAB.

VIII-E. Conclusión (Parte E)

La automatización de la puerta de garaje mediante el microcontrolador ATmega328P demostró una respuesta robusta y segura ante todas las condiciones de operación. La implementación de la Máquina de Estados Finitos en ensamblador garantizó un control determinista sobre las salidas del motor y la alarma, mientras que el uso de la interrupción PCINT0 permitió priorizar la seguridad activa al reaccionar instantáneamente ante presencia de obstáculos sin perder ciclos de procesamiento en el bucle principal. Finalmente, la integración del puerto USART proporcionó un canal claro de supervisión en tiempo real del estado del sistema.

IX. BIBLIOGRAFÍA

- ATmega328P Datasheet:
<https://www.alldatasheet.com/html-pdf/313218/ATMEL/ATmega328P/312/2/ATmega328P.html>
- 1088AS Datasheet:
<https://www.alldatasheet.com/datasheet-pdf/pdf/1179268/ETC/1088AS.html>
- Documentos y presentaciones vistas en clase.

X. LINK GITHUB Y VIDEOS DE DRIVE

- Link Github:
<https://github.com/brunobentancourt/laboratorio2>
- Link Drive:
https://drive.google.com/drive/folders/1Nu3vLOcjOczx_-dxtzkbjCPuUe_KuI21